== vs ===

Why "0" == false is true

equality.js

```
1  console.log("0" == false); // true
2  console.log("0" === false); // false
```

output **true** **false**

Why it happens

01 == converts first
Loose equality changes the types, then compares the values.

02 Booleans become numbers
false becomes 0 and true becomes 1.

03 Strings become numbers
When compared with a number, "0" becomes 0.

04 === converts nothing
Strict equality checks type AND value. Different types means false.

Step by step

steps.js

```
1  // "0" == false
2  // step 1: false -> 0
3  // step 2: "0" -> 0
4  // step 3: 0 == 0 -> true
```

Guess the output

Classic loose equality questions

expression	result	why
`"" == 0`	true	"" becomes 0
`"1" == true`	true	both become 1
`null == undefined`	true	special rule
`null == 0`	false	null only equals undefined
`[] == false`	true	[] to "" to 0
`NaN == NaN`	false	NaN equals nothing

Traps to mention

- **"0" == false is true, but if ("0") still runs.** A non-empty string is truthy.
- **The one accepted use of ==:** `x == null` checks null and undefined together.
- **Default to ===.** Most style guides and linters enforce it.

Your 20-second answer

"Double equals does type coercion before comparing. false becomes 0 and the string '0' becomes 0, so they are equal. Triple equals checks type and value without converting, so a string and a boolean are never equal. I always use triple equals."